

Problem Set 2: Solutions

Tokens & Regular Expressions (Lecture 2)

Compiler Construction Module

1 Classify the tokens

Problem: Classify every token in `while (n >= 1.0) total = total + n;`

Solution

Twelve tokens, in order:

#	token text	class	note
1	<code>while</code>	keyword	reserved word, not an identifier
2	<code>(</code>	punctuation	
3	<code>n</code>	identifier	
4	<code>>=</code>	operator	one token, not <code>></code> then <code>=</code> : longest match
5	<code>1.0</code>	float	the dot and the fraction make it a float
6	<code>)</code>	punctuation	
7	<code>total</code>	identifier	
8	<code>=</code>	operator	
9	<code>total</code>	identifier	
10	<code>+</code>	operator	
11	<code>n</code>	identifier	
12	<code>;</code>	punctuation	

The spaces produce no tokens. There is no integer token in the line: `1.0` is a float literal.

2 Track the position

Source (1-based lines and columns, each line ends with a newline):

```
sum = 0
count = 10
sum = sum + count
```

Problem: 2(a): position of `count` on line 2

Solution

Line 2, column 1. The newline that ends line 1 resets the column counter to 1 and increments the line counter; `count` is the first character of the new line.

Problem: 2(b): position of + on line 3

Solution

Count the characters of line 3: `s=1`, `u=2`, `m=3`, `space=4`, `=5`, `space=6`, `s=7`, `u=8`, `m=9`, `space=10`, `+=11`. The lexer reports **line 3, column 11**. (The ten characters of `sum = sum` precede it.)

Problem: 2(c): why track position while scanning

Solution

Because position is a property of *where the token started*, and the only moment the lexer knows that cheaply is when it is there. Tracking is one increment per character read (plus “line++, column=1” on a newline), and every token gets its start position for free, including tokens that follow skipped whitespace and comments, which leave no trace in the token stream. Computing positions afterwards would require either keeping the whole source in memory and re-scanning it from the beginning for every token (quadratic work) or storing byte offsets and converting them later, which still means re-reading the text to count newlines. Error messages, debuggers and IDEs need line and column for every token, so the by-hand count is the right design.

3 Find and fix the bug

```
StringBuilder sb = new StringBuilder();
while (true) {
    int c = input.read();
    sb.append((char) c);
    if (c == '"') break;
}
return new Token(TokenType.STRING, sb.toString());
```

Problem: 3(a): trace on the input `hi"`

Solution

iteration	c	sb after append	test c == '"'
1	h	h	false
2	i	hi	false
3	"	hi"	true: break

The returned token contains **three** characters, `hi"`: the closing quote is part of the content.

Problem: 3(b): the bug

Solution

The code appends the character *before* checking whether it is the closing delimiter, so the delimiter is appended too. Delimiters tell the lexer where the string ends; they are not part of its value. There is a second, worse problem hiding in the same loop: if the input ends before a closing quote, `read()` returns `-1` forever, `(char) -1` is appended forever, and the loop never terminates.

Problem: 3(c): corrected code

Solution

Test first, append second, and handle end of input:

```
StringBuilder sb = new StringBuilder();
while (true) {
    int c = input.read();
    if (c == -1)
        throw new LexicalError("unterminated string literal");
    if (c == '"') break;           // delimiter: stop, do not
    // append
    sb.append((char) c);
}
return new Token(TokenType.STRING, sb.toString());
```

On `hi"` the content is now `hi`. (A full lexer would also handle escape sequences such as `\"` inside the loop, by reading the next character and appending its decoded value instead of stopping.)

4 Write the regex

Abbreviations: `digit` = `[0-9]`, `hexdigit` = `[0-9a-fA-F]`.

Problem: 4(a): hexlit

Solution

```
hexlit = "0x" {hexdigit}+
```

The literal two characters 0x, then one or more hex digits (+, not *: 0x alone is not a number).

Problem: 4(b): two matches, two near-misses

Solution

Matches: 0x1A3F, 0xff (also 0x0, 0xDEADbeef).

Does not match: 0x (no digits after the prefix), 1A3F (no prefix), 0xG1 (G is not a hex digit; the lexer would stop after 0x, which itself is not a token, so this is a lexical error or lexes as 0 then identifier xG1 depending on the other rules), 0x 1 (the space breaks the literal).

Problem: 4(c): capital X

Solution

No: "0x" is a literal string and matches a lowercase x only. Revised:

```
hexlit = 0[xX] {hexdigit}+
```

The character class [xX] accepts either letter. (Equivalently ("0x" | "0X") {hexdigit}+.)

5 Put it together

Problem: 5(a): a JFlex fragment for identifier = integer ;

Solution

```
/* abbreviations */
digit  = [0-9]
letter = [a-zA-Z_]
ident  = {letter} ({letter} | {digit})*
intlit = {digit}+
ws     = [ \t\r\n]+

%%

<YYINITIAL> {
  {ws}      { /* skip whitespace: no token */ }
  "="      { return new Token(TokenType.ASSIGN, yytext(), yyline
    , yycolumn); }
```

```

";"      { return new Token(TokenType.SEMI,  yytext(), yyline
        , yycolumn); }
{intlit} { return new Token(TokenType.INT, Integer.parseInt(
        yytext()), yyline, yycolumn); }
{ident}  { return new Token(TokenType.ID,  yytext(), yyline,
        yycolumn); }
.        { throw new LexicalError("unexpected character '" +
        yytext() + "'", yyline, yycolumn); }
}

```

The last rule catches anything else as an error instead of silently dropping it.

Problem: 5(b): trace on `count = 42;`

Solution

input consumed	rule (longest match)	emitted
count	{ident} (5 chars; stops at the space)	ID("count")
space	{ws}	nothing
=	"="	ASSIGN
space	{ws}	nothing
42	{intlit} (2 chars; stops at ;)	INT(42)
;	";"	SEMI

Token stream: ID("count") ASSIGN INT(42) SEMI.

Problem: 5(c): would ending lex as end + ing?

Solution

No. A lexer generator applies two tie-breaks in a fixed order. **First, longest match:** at the start of `ending`, the keyword rule `"end"` matches 3 characters but `{ident}` matches all 6, so the identifier wins outright and the token is `ID("ending")`; rule order is never consulted. **Second, rule order, only on equal length:** that applies when the input is exactly `end` (followed by a non-identifier character): both rules match 3 characters, and the one written first in the spec wins, so the keyword rule must be placed *before* `{ident}` to get `END` rather than `ID("end")`. Priority can never split a longer identifier into a keyword plus a remainder, because length is decided before priority is looked at.